기존 대사 시스템을 AI 에이전트 개발의 학습 대상으로 삼았습니다

소규모 사업자를 위한 Java 프로젝트를 실제로 확장하며 작업 방식을 익혔습니다 .

출발점

이미 개발한 잔액 대사 시스템

날짜별 예정 금액과 은행 입출금 비교

학습 목표

AI 에이전트를 구성하고 개발 · 검토를 연결하기

기존 코드를 보존하면서 미수금 관리로 확장

코드 생성뿐 아니라 작업 분배 → 실행 → 검토 → 수정까지 경험하는 것이 목적

왜 Claude Code 가 아닌 Codex 를 선택했나

선택 기준은 이번 프로젝트에 연결된 작업 환경과 역할 분담을 활용한 학습이었습니다 .

필요했던 작업	Codex 에서 활용한 방식	프로젝트에 맞았던 이유
기존 코드 ·DB 파악	저장소와 실행 환경에서 함께 확인	처음부터 다시 만들지 않아도 됨
역할별 작업 분담	총괄 · 하위 역할로 파일 범위 배분	조사 결과를 구현과 검토에 연결
결과를 실행으로 확인	수정 · 테스트 ·Git 기록 연결	설명과 실제 동작을 함께 확인

선택의 근거 : 이미 연결된 Codex 작업 환경 + 총괄 · 하위 역할을 활용한 개발 흐름

총괄 아래 기획 · 조사 · 디자인 · 개발 · 검토 역할을 구성했습니다

제가 목표와 업무 기준을 정하고 , 총괄 에이전트가 작업 배분과 결과 통합을 맡았습니다 .



분석 결과 → 범위 · 설계 → 기능별 구현 → 독립 검토 → 수정 · 재검증

독립 작업은 병행하고 , 선행 결과가 필요한 작업은 순서대로 진행

작업 결과는 청구 · 입금 · 배분을 관리하는 서비스로 확장됐습니다

기존 대사 기능을 유지하고 , 사업체별 미수금 모델과 모바일 화면을 추가했습니다 .

모바일 · 웹 화면

TypeScript · React Native
Expo · React

→

Spring Boot API

Java 21 · Spring Security
세션 · 사업체 권한 검사

→

PostgreSQL 17

원본 · 배분 · 취소 · 감사
Flyway V1–V13

Controller → Service → JdbcTemplate(SQL) → DB

핵심 데이터 관계

입금 1 : N 배분 라인 N : 1 청구

부분 · 묶음 입금을 표현

기존 대사 JPA 구조 유지 | 빌드 · 관리 : Gradle · Git/GitHub

핵심 개발 01–02 | 계정과 거래 원본을 관리합니다

서버의 사업체 권한 검사부터 청구 · 입금 등록까지

01 계정 · 사업체 접근 제어



인증과 소유권

세션과 멤버십을 서버에서 확인

사업체별 데이터

다른 사업체 자료 접근 차단

등급 · 사용량

성공한 후보 실행만 집계

02 청구 · 입금 등록



청구 조건

거래처 · 금액 · 지급기한 입력

입금 원본

입금자 · 날짜 · 금액 별도 기록

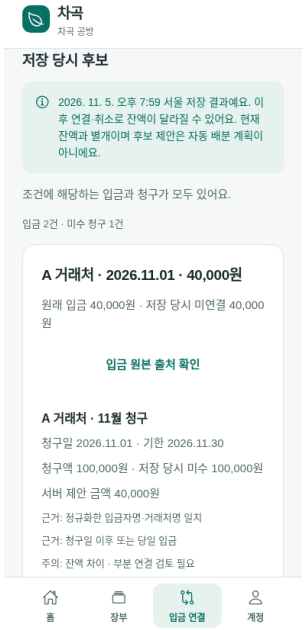
재요청 처리

같은 키의 요청은 결과 재사용

핵심 개발 03–04 | 후보를 검토하고 연결 금액을 확정합니다

추천과 확정을 분리해 현재 잔액을 기준으로 처리합니다 .

03 규칙 기반 매칭 후보



조회 범위

선택한 기간의 거래를 조회

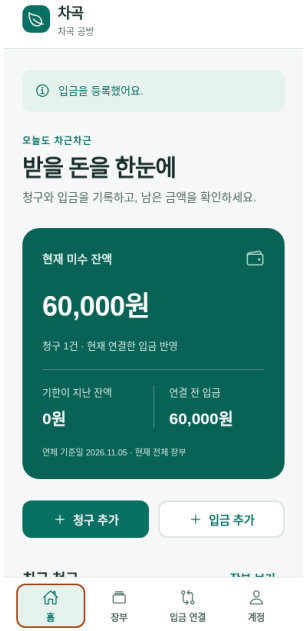
후보 근거

날짜 · 금액 · 입금자명으로 제안

검토 단계

후보 생성만으로 배분하지 않음

04 배분 · 미수금 계산



부분 · 묶음 연결

여러 입금과 청구에 금액 배분

확정 직전 검사

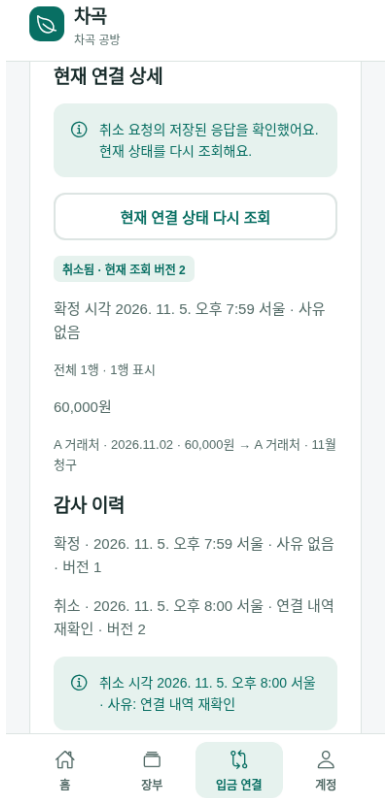
부모 행 잠금 후 잔액 재검사

서버 계산 결과

확정된 금액만 미수에 반영

핵심 개발 05 | 연결 취소와 감사 이력을 남깁니다

입금 원본과 확정 기록을 보존하면서 취소된 배분을 잔액 계산에서 제외합니다 .



취소 요청에 남기는 정보

누가 · 언제 · 어떤 사유로 취소했는가

원본 보존 입금 · 청구 · 기존 배분 라인을 삭제하지 않음

잔액 복원 취소한 연결을 제외해 미수금을 다시 계산

한 번에 저장 취소 기록 · 감사 · 명령 결과를 함께 커밋

시연 영상 보기 ↗

독립 검토는 실제 수정으로 이어졌습니다

저장 후 응답이 끊긴 상황에서 발견한 중복 저장 위험

DB 저장은 성공 → **공백만 수정해 재시도** → **서버는 다른 명령으로 판단**

화면은 응답을 받지 못함 / 요청 키가 새로 만들어짐 / 중복 저장 가능성 발생

수정 **정규화한 내용이 같으면 기존 요청 키 유지**

검증 **저장 후 503 응답을 재현 → 같은 키 재시도 → 입금 1 건 확인**

직접 설치한 Hermes 에는 읽기 전용 조회 도구를 연결했습니다

에이전트 활용에서 도구 입력 · 권한 · 응답 범위를 직접 확인하는 실습으로 이어갔습니다 .

Hermes
날짜를 받아 도구 호출

→

Python 도구
인증된 GET 만 실행

→

Java API + DB
현재 원장 금액 반환

확인한 응답

청구액	확정 배분	미수금
100,000 원	**40,000 원**	**60,000 원**

현재 원장 요약과 API 경로를 근거로 설명 · 제품 FREE 기능과 별도 실습

기존 기능을 보존하고 , 검토 가능한 결과를 남겼습니다

기존 시스템 재사용	Java · PostgreSQL · 기존 대사 API 유지
기능별 검토 · 수정	독립 검토의 재현 절차를 구현에 반영
개인 PC 실행 확인	환경 구성부터 전체 테스트 · 도구 호출까지 수행

검증 근거

백엔드 186 개 통과

11.04 · 사용자 PC / 실패 · 오류 · 스킵 0

클라이언트 13 개 · 브라우저 9 + 9 개

11.01 · 기존 에이전트 실행 기록

업무 규칙을 정하고 , 실패 뒤의 데이터까지 확인하는 법을 배웠습니다

```
assertMoney(paymentBalance(owner, p1), "unallocatedAmount", "60");
```

같은 키로 금액을 바꾼 요청이 거절된 뒤 , 미배분 잔액 60 원이 유지되는지 검사

직접 수행한 일

목표 · 업무 기준과 에이전트 역할 구성
안내에 따른 assertion 작성 · 검증 위치 설명
환경 설치 · 테스트 · Hermes 예제 수정과 실행

배운 점

요청 키와 동시성 문제를 구분
원본 보존과 배분 취소를 구분
조회 범위를 넘는 AI 설명을 경계